

Description of the Simulation Code Structure for Speech Enhancement

This section provides the implementation and simulation codes for the proposed speech enhancement method based on sparse representation and dictionary learning. The folder structure and the order of file execution are described below:

1. Initial Data Folders

- **Clean_Train:** Contains clean speech audio files used for training the algorithm.
 - **Clean_Test:** Contains clean speech audio files used for testing and evaluating the algorithm.
-

2. Essential Folders for Proper Code Execution

To ensure proper execution in **MATLAB**, please add the following folders to your MATLAB path before running any file. Use the path:

Home > Set Path > Add with Subfolders

- **AUX_FUNCTION:** Contains auxiliary functions such as Voice Activity Detection (VAD), signal reconstruction, and other required utilities.
 - **HPR_main:** Includes implementations of sparse representation algorithms such as **Lasso**.
 - **Speech-measure-SDR-SAR-STOI-PESQ-master:** Contains signal quality evaluation metrics like **PESQ**, **STOI**, **SDR**, and **SAR**.
 - **ksvdbox** and **ompbox:** Contain functions for **dictionary training (K-SVD)** and **sparse coding (OMP)**.
-

3. Structure and Execution Order of Main Files

After ensuring that all folders are correctly added to the MATLAB path, the following files should be executed in order to run the algorithm step-by-step:

- **Predefined noise files and dictionaries:**
 - Includes 10 common noise types (e.g., street noise, babble, white noise, etc.) stored in .wav format.

- Trained dictionaries for clean speech and each noise type are also available and stored in files.
 - **CreateDATA_1.m:**
 - Reads files from the **Clean_Train** folder and performs **STFT** on them.
 - The output is used as training data for the clean speech dictionary.
 - **CreateDATAnoise_2.m:**
 - Used to create training data for **noise dictionaries**.
 - For each noise type, the corresponding audio file is read and processed.
 - **KSVD_Dictionary_Learning_3.m:**
 - Trains the dictionaries using the data from the previous steps.
 - **Clean speech and noise dictionaries are trained separately.** Ensure file names are properly set and do not overlap.
 - **Final_enhancement_byNMIDL_4.m:**
 - This is the **main file for simulation and execution** of the speech enhancement algorithm.
 - Trained dictionaries for clean speech and noise are loaded, and the **proposed algorithm** is applied.
 - The user can specify: the **noise type**, **desired SNR**, and the **number of non-zero coefficients (sparsity level)**. The enhancement is then carried out automatically.
-

4. Important Note on Performance Evaluation

The results reported in the paper are the **average of multiple runs** on various speech signals. Since the test speech files are short and the noise files are long (several minutes), during each run, a **random segment of the noise** is selected and added to the speech at a given SNR.

Due to this random selection, the results may vary slightly in different runs on different test files (slightly higher or lower than the reported values). Therefore, the final

evaluation of the algorithm's performance is based on the **average results over multiple full runs of the Final_enhancement_byNMIDL_4.m file**, where each run is averaged over **10 different speech files**.

5. Recommendation for Fast and Efficient Use

Given the time-consuming nature of training dictionaries for clean speech and 10 types of noise, and since the trained dictionaries are already provided in this project, it is **recommended to directly run the Final_enhancement_byNMIDL_4.m file** after setting the MATLAB path, to quickly evaluate the performance of the method.
